TCP: Segment, Sequence Numbers, Retransmission

Lecture 17

Computer Networks and Distributed Programming

Where we are: the protocol that is neither

The mechanisms are all in place — checksum, sequence numbers, acknowledgements, timers, a window. Now they get a header.

- TCP acknowledges cumulatively, as Go-Back-N does, and keeps what arrives early, as selective repeat does.
- Today: what a segment carries, what the two numbers in it mean, and how the sender decides that something was lost.
- One decision runs through all of it — the numbers count bytes, not packets.
- The opening handshake, the receive window and congestion are in the same header, and come after.

What a TCP connection is

- Connection-oriented: the two sides exchange preliminary segments and build state before a single byte of data moves.
- Reliable: detection, acknowledgements, timers, sequence numbers, retransmission: everything we have just built, in one protocol.
- Full-duplex: data flows from A to B at the same time as it flows from B to A, over the one connection.
- Point-to-point: one sender, one receiver. There is no TCP multicast — that ground belongs to UDP.
- Byte stream: the application hands over bytes, not messages, and TCP decides where the segment boundaries fall.

A connection with nothing in the middle

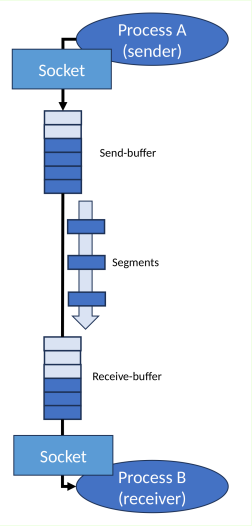
A logical connection

The two transport modules behave as if a reliable channel joined them directly. Underneath, the segments cross a network that only routes.

- It is not a circuit: no capacity is reserved, and nothing is set aside for the connection inside the network.
- Buffers, variables and timers exist in the two end systems and nowhere else.
- Routers see datagrams, not connections: they forward the segment without reading a byte of its header.
- Which is why reliability is built here. The network layer is busy deciding where a packet goes, not whether it arrived.

Send buffer, receive buffer

- `write` does not transmit: it copies bytes into the send buffer, and TCP sends them from there at its own convenience.
- `read` does not receive: it takes what is already sitting in the receive buffer, put there by earlier segments.
- The two buffers absorb the mismatch between how fast the application writes, how fast the network moves, and how fast the other side reads.
- Each end of the connection has one of each, because each end both sends and receives.



How much data fits in one segment

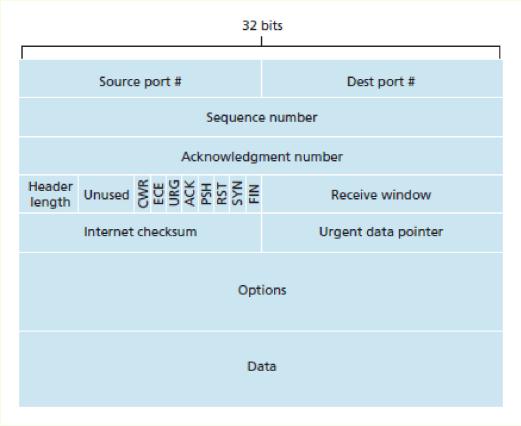
Maximum segment size

The MSS is the largest amount of application data TCP will put in one segment. It is not the size of the segment.

- It is chosen so that the segment, plus the TCP and IP headers, fits in one link-layer frame.
- That frame has a maximum size of its own, the MTU: 1500 bytes on Ethernet.
- With the usual $20 + 20$ bytes of header, $1500 - 40 = 1460$ bytes of data per segment.
- A three-megabyte image is not a message: it is two thousand segments, and the application never learns where the cuts fell.

The TCP segment

- Two port numbers, as in UDP: the segment is still demultiplexed onto a socket.
- Two 32-bit fields, sequence and acknowledgment number, which carry the reliable transfer.
- A 16-bit receive window: how many bytes the other side will accept.
- Header length in 32-bit words, because the options make the header variable.
- One checksum, computed as it was for UDP.



The flag bits

- ACK: the acknowledgment number field means something. Set on practically every segment after the first.
- SYN, FIN, RST: opening a connection, closing it, refusing it: three bits we will come back to.
- CWR, ECE: explicit congestion notification — the network tells the two ends it is congested, instead of just dropping.
- PSH: hand what you have to the application now. URG: this segment carries urgent data, and the pointer says where it ends.
- Eight flags, four unused bits beside them — and the last two flags are not used in practice.

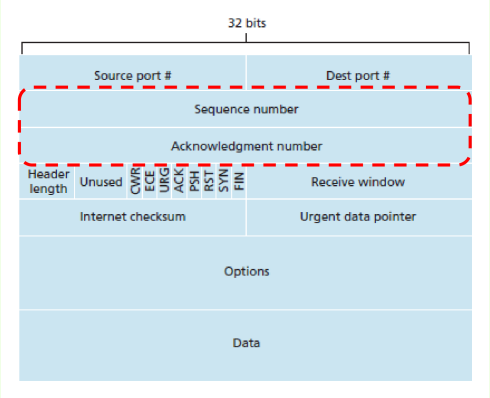
Numbers over bytes, not over segments

The two numbers

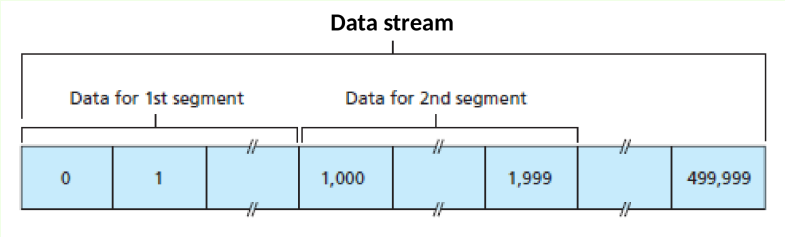
Sequence number: the position in the byte stream of the first byte of this segment's data.

Acknowledgment number: the position of the next byte that the sender of this segment expects to receive.

- TCP knows nothing about the data it carries: an ordered stream of bytes, and nothing else.
- So the numbering survives a different choice of where to cut the stream.

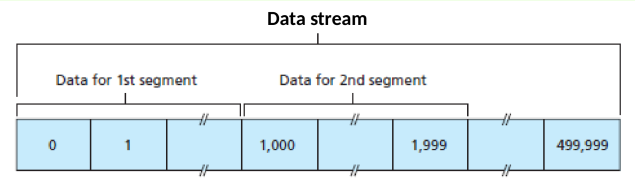


500,000 bytes, 500 segments



- A file of 500,000 bytes, an MSS of 1000 bytes, and a first byte numbered 0.
- TCP builds 500 segments, with sequence numbers 0, 1000, 2000, and so on: one every MSS bytes.
- The second one is not “segment number one”. It is the segment whose first byte is the thousandth.

The acknowledgment number is what I expect next



- B receives bytes 0 to 999 and answers with acknowledgment number 1000: not what arrived, but what comes next.
- Then bytes 1000 to 1999, and the answer is 2000. The number is always the first byte B does not have.
- It exists before any data does: as soon as the connection is open, B is already waiting for a specific byte.

Cumulative acknowledgements

One number for everything received

A has bytes 0–535 from B, and then a segment with bytes 900–1000. Bytes 536–899 never arrived. A acknowledges 536.

- An acknowledgement says *everything below this number has arrived*, and says nothing at all about what is above it.
- So one acknowledgement covers any number of segments, and losing one costs nothing as long as a later one gets through.
- There is no way to report the hundred bytes A is holding: the header has one number, and the gap comes first.
- That single number is what makes the sender cheap — and what makes a missing segment expensive.

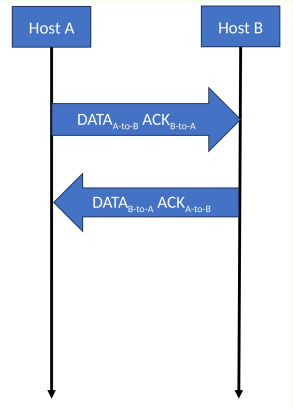
A segment that arrives too early

Bytes 900 to 1000 are here, and 536 to 899 are not. Keep them, or throw them away?

- The RFCs do not decide. Both behaviours are correct, and the choice is left to whoever writes the implementation.
- Throw away: the receiver stays simple — no buffer, no bookkeeping — and the sender pays by sending those bytes again.
- Keep: the bytes wait until the gap is filled, and the network carries each byte once.
- Every implementation in use keeps them, which is why one loss costs one retransmission and not a windowful.

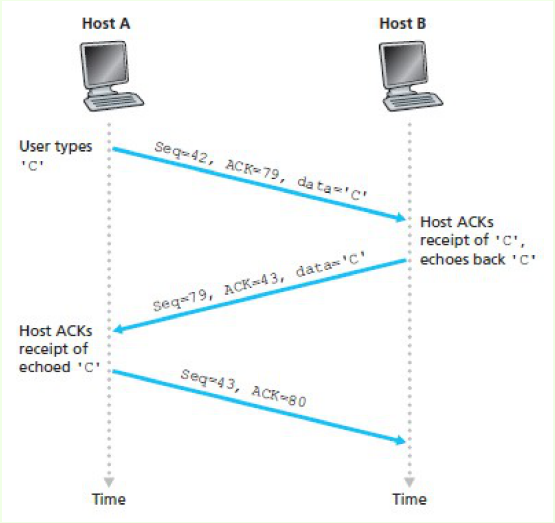
Both directions at once

- One connection, two byte streams: A to B and B to A, numbered independently of each other.
- A segment leaving A carries a sequence number for the A-to-B stream and an acknowledgment number for the B-to-A stream.
- The two fields in one header therefore speak about two different streams.
- An acknowledgement travelling inside a segment that also carries data is said to be piggybacked: it costs no extra packet.



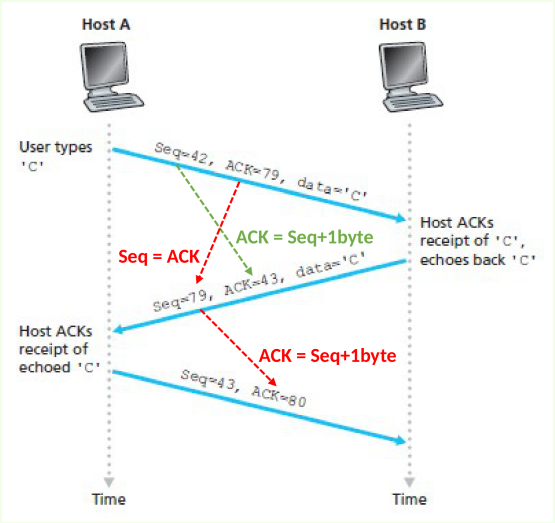
One character, three segments

- A remote login: every character typed at A is sent to B, and echoed back so that the user sees what arrived.
- The two streams start at 42 and 79, agreed when the connection was opened — not at zero.
- The user types one letter and goes for a coffee. Three segments cross the network, and each carries both numbers.



The same exchange, read as two flows

- A to B: seq 42, where the letter sits; ack 79, the byte A expects from B and has not seen yet.
- B to A: seq 79, where the echoed letter sits; ack 43, one past the byte that just arrived.
- A to B: nothing left to send: seq 43, ack 80. A pure acknowledgement, with an empty data field.
- Green follows the A-to-B stream, red the B-to-A one. Each answer is seq + 1 of the flow it acknowledges.



A segment with no data still has a sequence number

The third segment carries nothing. What is seq 43 doing there?

The field is in the header whether it is used or not, so it says where A would start if it had anything to send. It consumes no byte: A's next segment will carry 43 again.

- A pure acknowledgement does not advance the stream, and what B is waiting for does not move: still byte 43.
- Which also means nobody acknowledges it. There is nothing to acknowledge, and no timer is started for it.
- If it is lost, nothing is retransmitted: the next acknowledgement A sends will carry the same information, or more.

Where the numbering starts

- In every example so far the first byte was numbered 0. In a real connection it never is.
- Each side picks its initial sequence number at random, and tells the other one while the connection is being opened.
- The reason is the one we met with sequence number reuse: a segment from an earlier connection can still be alive in the network.
- Same two hosts, same two ports, numbering from zero every time — and an old copy would be accepted as current data.
- A random start does not make that impossible. It makes it improbable, which is as far as the protocol can go.

The sender in three events

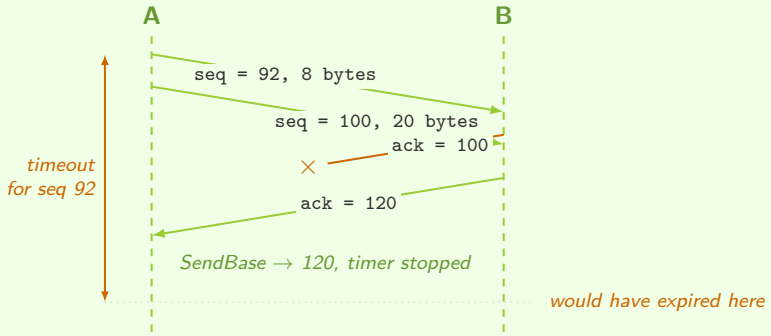
Event	What the sender does
Data arrives from the application	Build a segment with sequence number `NextSeqNum`, start the timer if it is not already running, pass it to IP, and advance `NextSeqNum` by the number of bytes sent.
The timer expires	Retransmit the unacknowledged segment with the smallest sequence number, and start the timer again.
An acknowledgement y arrives	If $y >$ `SendBase`, move `SendBase` to y, and restart the timer if anything is still unacknowledged.

Two variables hold the whole sender: `SendBase`, the oldest byte not yet acknowledged, and `NextSeqNum`, the next one to go out.

One timer, not one per segment

- A timer for every outstanding segment is the natural design, and it is what selective repeat asks for.
- TCP keeps one, attached to the oldest unacknowledged segment — the one `SendBase` points at.
- When it fires, only that segment goes out again, however many others are in flight and unanswered.
- So a sender with ten segments outstanding retransmits one, and lets the cumulative acknowledgements sort out the rest.
- The reason is cost: one deadline to track instead of one per segment, on a machine that may hold thousands of connections.

A cumulative acknowledgement that saves a retransmission



one acknowledgement was lost, and nothing was sent twice

The first acknowledgement was lost, and nothing was retransmitted: ack 120 says that everything below 120 arrived, which covers both segments at once. `SendBase` jumps straight to 120 and the timer is stopped.

Doubling the timeout

- When the timer expires, TCP retransmits and sets the next timeout to twice the interval that has just run out.
- Three quarters of a second becomes 1.5, then 3, then 6: the waits grow exponentially for as long as nothing comes back.
- The estimate is not used while this is going on. It comes back as soon as an acknowledgement arrives and the timer restarts normally.
- A timeout usually means congestion, and retransmitting harder into a congested network makes it worse.
- So the sender backs off. It is a crude form of congestion control, and the same idea returns when stations collide on a shared link.

How long should the timer be

- Too long: every loss is paid with a long idle wait, and the transfer stops for as long as the timer runs.
- Too short: segments that are merely slow get declared lost, and the network carries copies nobody needed.
- We have already seen the extreme: with a timeout shorter than the round trip, everything is retransmitted, losing nothing.
- So the timeout has to be larger than the RTT — and the RTT is not a constant, and nobody tells the sender what it is.

How do you measure something that changes while you measure it?

Sampling the round trip

SampleRTT

The time between passing a segment to IP and receiving an acknowledgement that covers it. One measurement, for one segment.

- TCP measures one segment at a time, not all of them: roughly one new sample per round trip.
- The samples move, and by a lot — router queues fill and drain, and the host at the other end has a load of its own.
- So a single sample is a poor estimate of the round trip, and a very poor basis for a deadline.
- And segments that have been retransmitted are never sampled at all.

Which transmission was that acknowledgement for?

A segment goes out, the timer fires, it goes out again — and then an acknowledgement arrives.

It carries a byte number, not a timestamp: it looks identical whether it answers the first copy or the second. The sender cannot tell.

- Measure from the first transmission, and the sample is far too long if the acknowledgement is really answering the second.
- Measure from the second, and it is far too short if the first copy got through and its acknowledgement was merely slow.
- A short sample shortens the timeout, which causes more premature retransmissions, which produce more short samples.
- The rule cuts the loop before it starts: never sample a segment that was retransmitted.

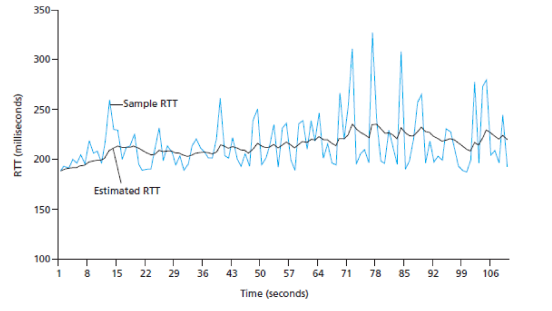
EstimatedRTT

Exponential weighted moving average

$$\text{EstimatedRTT} \;=\; (1-\alpha)\cdot\text{EstimatedRTT} \;+\; \alpha\cdot\text{SampleRTT}, \qquad \alpha = 0.125 = \tfrac{1}{8}$$

- Every new sample pulls the estimate an eighth of the way towards itself: one odd measurement cannot drag it far.
- The weight of an old sample decays by $7/8$ at each update — exponentially, which is where the name comes from.
- So the average forgets. What the path was doing a minute ago counts for almost nothing now, and that is the point.
- No history is stored: one variable, and two operations per acknowledgement.

How much the RTT moves



$$\text{DevRTT} \;=\; (1-\beta)\cdot\text{DevRTT} \;+\; \beta\cdot\left|\,\text{SampleRTT}-\text{EstimatedRTT}\,\right|, \qquad \beta = 0.25$$

A real connection between Massachusetts and the south of France: the samples jump by a hundred milliseconds from one to the next, while the estimate stays smooth. DevRTT is the same kind of average, taken over how far the samples land from it.

The timeout interval

Retransmission timeout

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

- The estimate sets the floor: below it, acknowledgements arrive after the timer has already fired.
- The deviation sets the margin — wide when the path is erratic, narrow when it is steady.
- So the timeout follows a link that starts behaving badly, without anyone having to choose a number.
- Before the first sample there is nothing to compute from, so the initial value is one second.
- And after a retransmission the formula is set aside: the interval doubles instead, until something comes back.

When the receiver sends an acknowledgement

What arrives	What the receiver does
In-order segment, everything below it already acknowledged	Wait up to 500 ms for the next one; if it does not come, acknowledge.
In-order segment, with one earlier segment still waiting to be acknowledged	Acknowledge both at once, with a single number.
Out-of-order segment, numbered higher than expected: a gap	Acknowledge immediately, repeating the number at the bottom of the gap.
A segment that fills the gap	Acknowledge immediately, up to the end of what is now contiguous.

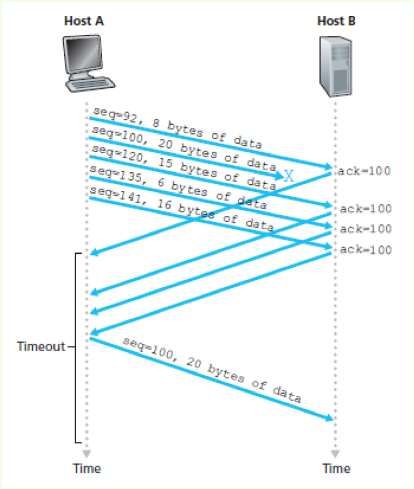
The first row is the delayed acknowledgement: two segments, one answer. The third row is where duplicates come from.

Duplicate acknowledgements

- A duplicate acknowledgement repeats a number the sender already had: taken alone, it carries no new information.
- But it was sent for a reason. Something arrived at the receiver, and it was not what the gap needed.
- So each duplicate is evidence that segments after the missing one are getting through: the path works, one segment does not.
- A sender that waits for its timer here is waiting for a deadline whose answer it already has.
- TCP has no negative acknowledgement. A repeated number is the closest thing to one, and it costs no new packet type.

Fast retransmit

- Five segments leave back to back. The one starting at byte 100 is lost; the other four arrive.
- The first is in order, and is acknowledged with ack 100. The other three land past the gap and repeat that same number.
- On the third duplicate the sender resends the segment at 100, without waiting for the timer.
- The timer was still running and would have fired much later: what is saved is most of a timeout interval.



Why three duplicates and not one

One duplicate already says that a segment is missing. Why wait for three?

Because one duplicate is exactly what reordering produces. Two segments swap places in the network, the receiver sends one duplicate, and the next arrival fills the gap by itself.

- Retransmitting on the first duplicate would send a copy of a segment that was never lost, and would do it often.
- Three in a row are hard to explain by reordering: three later segments got through while that one did not.
- The number is a compromise fixed by measurement rather than derived — three is what the standard settled on.

Go-Back-N or selective repeat?

- Like Go-Back-N: cumulative acknowledgements, no individual answer for what arrives early, one timer and two variables.
- Unlike Go-Back-N: a timeout retransmits one segment, not everything outstanding — and not even that, if the next acknowledgement covers it.
- Like selective repeat: the receiver buffers what arrives early instead of throwing it away.
- So TCP is neither: cumulative acknowledgements over a receiver that keeps things, which is the useful half of each.
- An option, selective acknowledgement, lets the receiver name the blocks it is holding — and moves TCP the rest of the way.

Summary

- A TCP connection is state in two end systems: buffers, variables, numbers. The network in between knows nothing of it.
- Sequence numbers count bytes; an acknowledgement names the next byte expected, and everything below it has arrived.
- The sender keeps one timer, on the oldest unacknowledged segment, and its length is estimated from the traffic itself.
- Three duplicate acknowledgements say that a segment is missing sooner than any timer can: fast retransmit.
- Still open: how the two ends agree on their initial numbers, and what the receive window in the header is for.